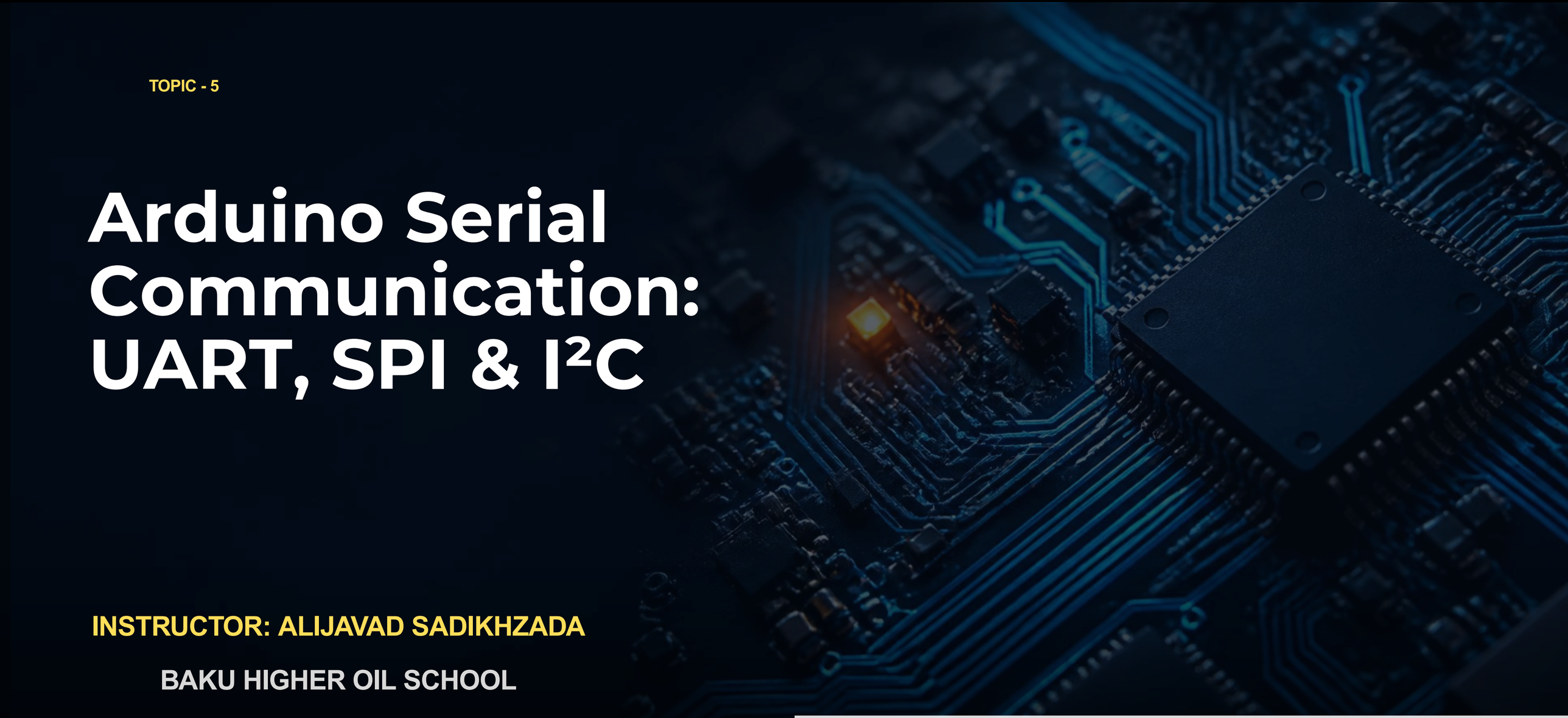
TOPIC - 5

# Arduino Serial Communication: UART, SPI & I²C

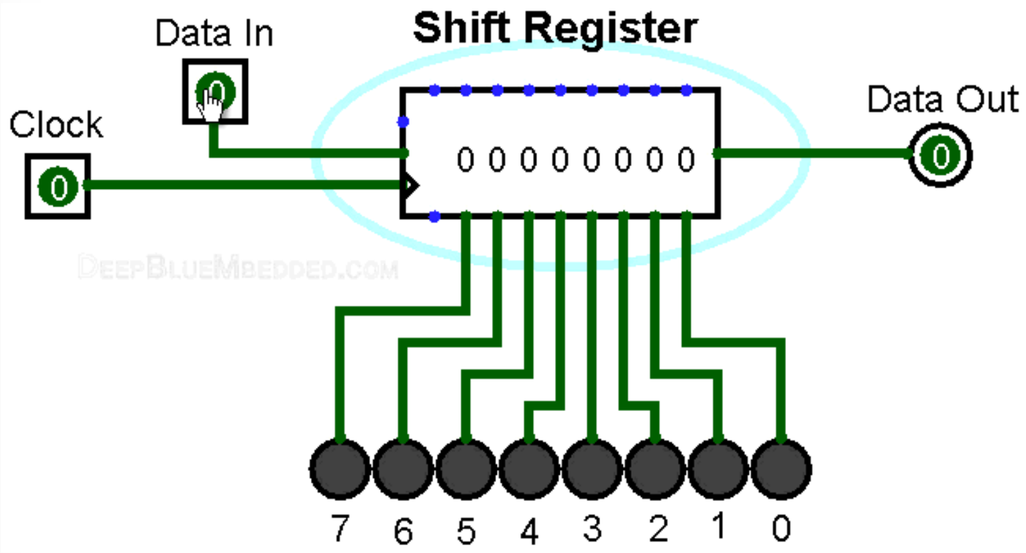



INSTRUCTOR: ALIJAVAD SADIKHZADA

BAKU HIGHER OIL SCHOOL

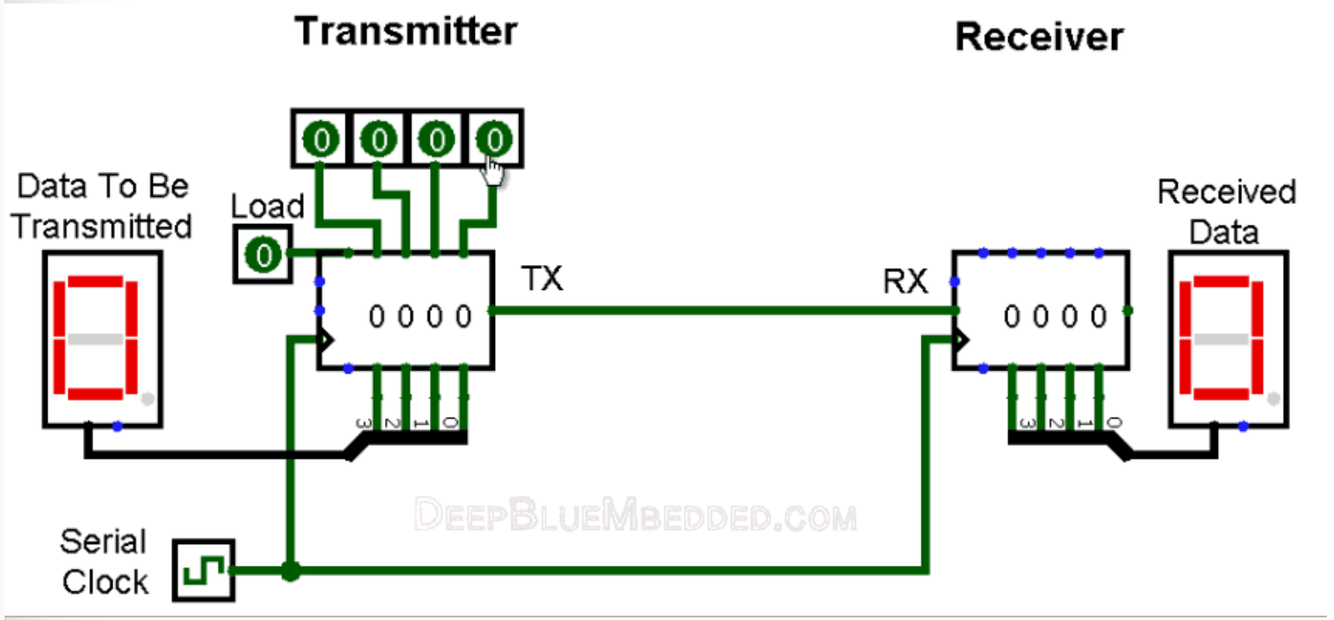

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Sends data one bit at a time, in sequence, over a single line
- Each bit takes one full clock cycle to transfer
- Parallel communication sends many bits per cycle, but needs many wires
- Serial runs at much higher clock frequencies, often beating parallel on real throughput
- Every serial protocol is built on the same core hardware: the shift register




# What is Serial Communication?

- A shift register is a chain of D-flip-flops sharing one clock line
- The transmitter's data output feeds directly into the receiver's data input
- Every clock pulse pushes one bit from transmitter to receiver
- Example: sending 4 bits takes exactly 4 clock cycles
- This simple shift-in/shift-out idea underlies UART, SPI, and I²C alike




# Shift Registers

- UART = Universal Asynchronous Receiver-Transmitter
- The most widely used serial protocol on embedded microcontrollers
- On Arduino, it's mainly used to talk to a PC through USB-to-TTL, powering the Serial Monitor
- Baud rate = how many bits are sent per second (bps)

UART is the protocol behind the Serial Monitor you already use for Serial.print() and Serial.begin(). It's the simplest of Arduino's three hardware serial buses: just two wires, no shared clock signal, and a speed setting — the baud rate — that both ends have to agree on ahead of time for the bits to line up correctly.

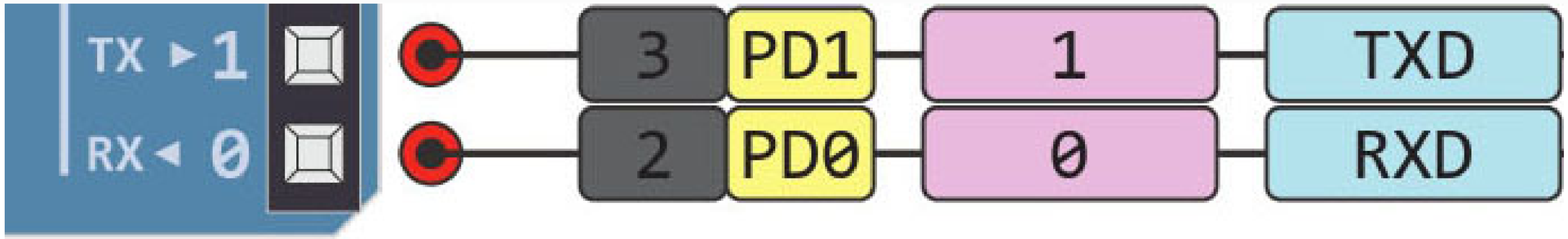
# Arduino UART - Overview

- UART (Asynchronous): no shared clock line at all
- Transmitter and receiver must be set to the same baud rate in advance
- USART (Synchronous/Asynchronous): adds an optional clock line
- The synchronous mode sends a clock signal alongside the data
- That shared clock keeps both sides perfectly synchronized, baud-rate guessing included

Plain UART is asynchronous: there's no clock wire, so both devices simply have to already agree on the bit rate and just trust the timing. USART is the more capable hardware variant that CAN also run synchronously, sending an explicit clock pulse with every bit so the receiver never has to guess timing at all — at the cost of an extra wire.

# UART vs USART

- Arduino UNO has one hardware UART module
- RX = pin 0 (receives incoming serial data)
- TX = pin 1 (transmits outgoing serial data)
- Need more UART ports? The SoftwareSerial library bit-bangs one on any 2 pins
- Software UART costs real CPU time and gets unreliable at high baud rates




# Arduino UART Pins

- Bluetooth modules (e.g. HC-05)
- WiFi modules driven by AT commands
- GSM / GPRS modules
- GPS modules
- Lidar sensors

UART shows up wherever a module ships its own simple command-and-data interface. Bluetooth boards like the HC-05, AT-command WiFi modules, GSM/GPRS cellular modules, GPS receivers, and Lidar distance sensors all default to a UART connection — it needs only 2 wires, and "send text, get text back" is about as simple as inter-chip communication gets.

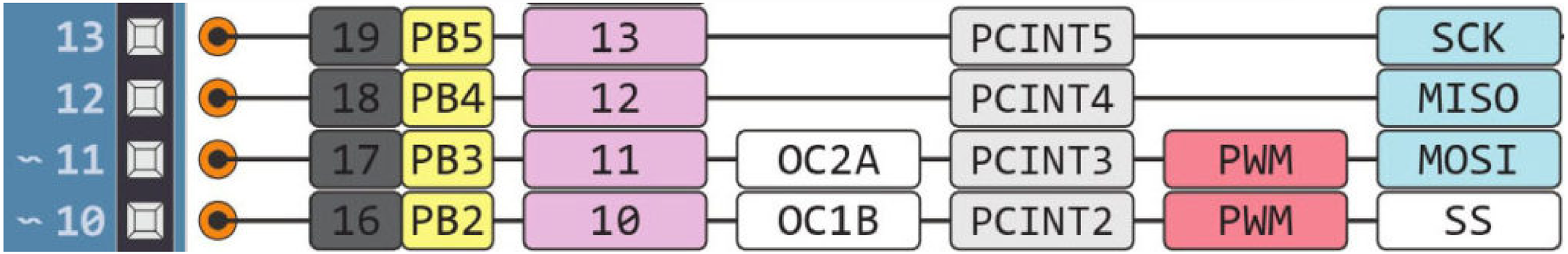
# Arduino UART Applications

- SPI = Serial Peripheral Interface
- Serial, synchronous, single-master, multi-slave, full-duplex
- Common for flash memory, ADC/DAC chips, RTCs, LCDs, and SD cards
- The master generates the clock and shifts data out while data shifts in — simultaneously
- Reading and writing both happen through that same shift-in/shift-out process

SPI trades pin count for raw speed: with a shared clock line, there's no baud rate to agree on and no ambiguity about timing. One master drives the clock and pushes data out to a slave while, on the very same clock edges, data shifts back in from that slave — a genuinely full-duplex exchange, which is why SPI tends to be the fastest of Arduino's three serial buses.

# Arduino SPI - Overview

- MOSI — Master Out, Slave In
- MISO — Master In, Slave Out
- SCK — Serial Clock, generated by the master
- SS — Slave Select, one dedicated pin per slave device
- More slaves = more SS pins needed — that's what limits bus size




# Arduino SPI Pins

- SPI TFT displays
- Dot-matrix LED displays
- External SPI flash memory chips
- External SPI ADC modules
- nRF24L01 wireless modules and SPI camera modules

SPI's raw speed makes it the default choice whenever a project needs to move real amounts of data quickly: refreshing a TFT or dot-matrix display, streaming to or from external flash or ADC chips, running an nRF24L01 wireless radio, or pulling frames from an SPI camera module all lean on that same fast, full-duplex shift-in/shift-out exchange.

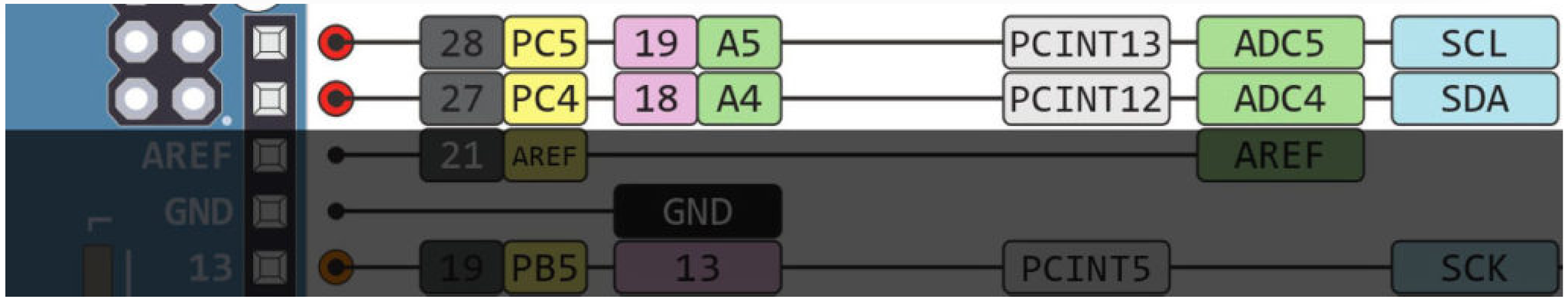
# Arduino SPI Applications

- I²C (I2C / IIC) = Inter-Integrated Circuit
- Needs only 2 wires total, no matter how many devices share the bus
- Multi-master, multi-slave — many devices, one shared pair of wires
- Every device on the bus has its own unique address
- The most IO-pin-efficient serial bus for connecting many devices at once

I²C is built for exactly the situation SPI struggles with: lots of devices on one bus without burning a pin per device. Every device — sensor, display, memory chip — shares the same two wires and is told apart purely by its own address, and because it supports multiple masters as well as multiple slaves, several microcontrollers can even share the same I²C network.

# Arduino I²C - Overview

- Arduino UNO has one hardware I²C module
- SDA (serial data) = pin A4
- SCL (serial clock) = pin A5
- Using I²C disables A4/A5 as analog (ADC) inputs
- Adding a new I²C device needs zero new pins — just tap into SDA/SCL




# Arduino I²C Pins

- RTC (real-time clock) chips
- OLED display screens
- I²C IO expander chips
- Temperature / humidity sensors
- External EEPROM memory, and the MPU6050 IMU (gyro + accelerometer)

Typically a project needs just one I²C bus on the whole board, no matter how many sensors and modules hang off it. RTC clock chips, OLED screens, IO expanders, temperature/humidity sensors, external EEPROM, and IMUs like the MPU6050 all happily share the same SDA/SCL pair — unlike UART's strict single-master, single-slave wiring.

# Arduino I²C Applications

| Feature | UART | I²C | SPI |
|---|---|---|---|
| Full Name | Universal Async. Receiver-Transmitter | Inter-Integrated Circuit | Serial Peripheral Interface |
| Pin Names | TX, RX | SDA, SCL | MOSI, MISO, SCK, SS |
| Type | Asynchronous | Synchronous | Synchronous |
| Clock Signal | No | Yes (master-generated) | Yes (master-generated) |
| Masters | 1 only | Multi-master | 1 only |
| Slaves | 1 only | Multi-slave | Multiple (1 SS pin each) |
| Typical Data Rate | Up to 115200 bps (2Mbps max) | 100kbps–3.4Mbps | Up to ~50Mbps |
| Distance | Several meters | Short — limited by bus capacitance | Short — several cm |
| Addressing | None needed (point-to-point) | 7-bit / 10-bit address per device | One SS pin per device |

# UART vs I²C vs SPI – At a Glance

- TX/RX pins use TTL logic levels — either 5V or 3.3V, depending on the board
- Never wire an Arduino TX/RX pin directly to a real RS232 port
- RS232 swings to roughly ±12V — more than enough to damage the pin
- Bridging TTL and RS232 needs a dedicated level-shifter chip (e.g. MAX232)
- Always check both devices' logic-level voltage before wiring them together

Two devices that both speak UART don't automatically speak the same voltage. Arduino's TX and RX pins are TTL-level — 5V on classic boards, 3.3V on many newer ones — and that's compatible with most sensor modules and other microcontrollers at the same voltage. A genuine RS232 port, the kind still found on older industrial equipment, is a different story: it swings between roughly +12V and -12V, well outside what a TTL pin can survive, so connecting it directly will damage the board. A MAX232-style level-shifter IC exists specifically to translate between the two safely.

# UART Logic Level

- Baud rate = how fast bits are sent, in bits per second (bps)
- Common values: 9600, 19200, 38400, 57600, 115200
- Both devices must be set to the exact same baud rate to understand each other
- Data bits = how many bits make up each transmitted byte
- Most common: 8 data bits; Arduino also supports 5, 6, 7, or 9

Baud rate and data-bit count are the first two settings both ends of a UART link have to agree on before anything useful happens. The baud rate sets the raw bit rate — 9600 bps is the classic Arduino default, though anything up to 115200 bps and beyond is common — and if one side is set differently from the other, every byte gets misread. Data bits set the frame size for each byte; 8 is essentially universal, covering a full ASCII byte, though the hardware can be configured for 5 through 9 bits for more specialized protocols.

# UART Baud Rate & Data Bits

# Parity, Stop Bits & Flow Control

- Parity = a simple error-checking bit added to each byte
- Even parity: the parity bit makes the total number of 1-bits even
- Odd parity: the parity bit makes the total number of 1-bits odd
- Stop bit(s) mark the end of a data frame — usually 1 bit (sometimes 1.5 or 2)
- Flow control (hardware RTS/CTS or software XON/XOFF) prevents buffer overflow

None of these settings change as often as baud rate, but all still have to match between devices for reliable communication. Parity adds one extra bit per byte purely for error-checking — even or odd — though “none” is by far the most common setting in simple Arduino projects. The stop bit (or bits) simply marks where one byte's frame ends, almost always just one. Flow control exists to stop a fast sender from overrunning a slow receiver's buffer, either through dedicated RTS/CTS wires or special XON/XOFF characters sent in-band — features most beginner Arduino sketches never need to touch.

- Serial.begin(baudRate); starts the UART module at the chosen speed
- Called once, inside setup()
- Serial.available() returns how many bytes are waiting in the input buffer
- Returns 0 when nothing has arrived yet
- Always check `available() > 0` before calling `read()`

```arduino
void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    // data is waiting -- safe to read it here
  }
}
```

# Serial.begin() & available()

- Reads one byte at a time from the input buffer
- Returns -1 if nothing is available

```ino
int incomingByte = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    incomingByte = Serial.read();
    Serial.print("I received: ");
    Serial.println(incomingByte, DEC);
  }
}
```

Serial.read() pulls exactly one byte off the UART input buffer and returns it as an int — or -1 if the buffer is empty, which is why it's almost always called right after checking Serial.available(). This sketch echoes back whatever byte arrives: read() grabs it into incomingByte, and Serial.println(incomingByte, DEC) prints its decimal value back out. It's the simplest possible receive loop, and the pattern — check available(), then read() — shows up in essentially every Arduino sketch that listens over UART.

# Serial.read() in Practice

- Serial.readBytes(buffer, length) reads a fixed number of bytes into an array
- Good for fixed-length packets of known size
- Serial.readBytesUntil(character, buffer, length) reads until a terminator byte shows up
- Common terminator: newline ('\n')
- Both return the number of bytes actually placed into the buffer

```
// Read exactly `length` bytes into buffer
size_t n = Serial.readBytes(buffer, length);

// Read into buffer until '\n' or `length` bytes
size_t n = Serial.readBytesUntil('\n', buffer, length);
```

# readBytes() & readBytesUntil()

- Serial.print(val) sends val as human-readable text — no newline after it
- Serial.println(val) does the same, then adds a newline
- Serial.write(val) sends the raw byte value — not its text representation
- print() takes an optional format: BIN, OCT, DEC, HEX, or decimal places for floats

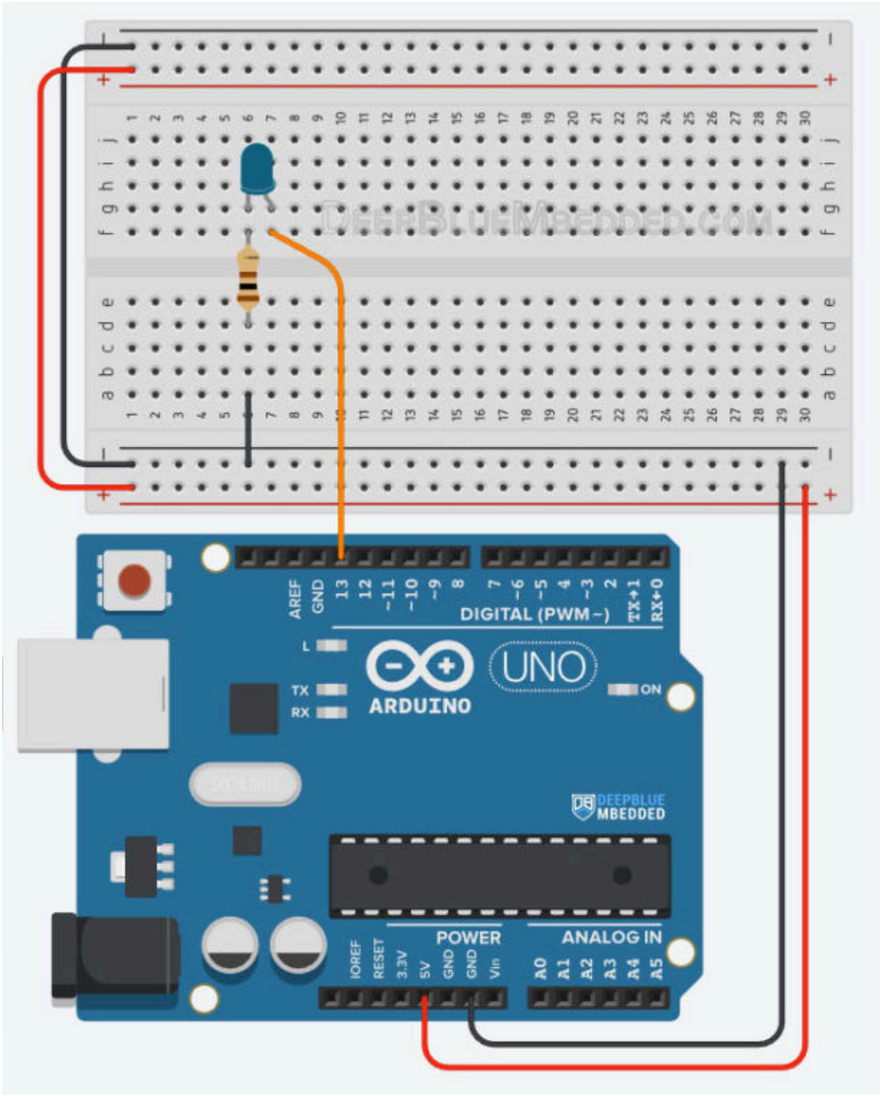
```
1   Serial.print(78, BIN);     // "1001110"
2   Serial.print(78, HEX);     // "4E"
3   Serial.print(1.23456, 2); // "1.23"
4
5   Serial.write(45);          // sends the single byte 45
6   Serial.write("hello");     // sends 5 raw bytes
```

# print(), println() & write()

| Function | What It Does |
| --- | --- |
| if(Serial) | Checks whether the Serial connection is active |
| Serial.availableForWrite() | Bytes free in the outgoing write buffer |
| Serial.end() | Shuts down the UART port and frees its pins |
| Serial.find() | Searches incoming data for a given string |
| Serial.findUntil() | Like find(), but stops at a terminator first |
| Serial.flush() | Waits until all outgoing data has finished sending |
| Serial.parseInt() | Reads the next integer out of the incoming text |
| Serial.parseFloat() | Reads the next floating-point number out of the incoming text |
| Serial.peek() | Returns the next byte without removing it from the buffer |
| Serial.readString() | Reads incoming chars into a String until timeout |
| Serial.readStringUntil() | Reads incoming chars into a String until a terminator |
| Serial.setTimeout() | Sets how long read-family functions wait for data |

# More Serial Library Functions

- Goal: 2-way communication between Arduino and PC over UART
- Arduino continuously reports the LED's current state
- The PC sends '1' to turn the LED ON, '0' to turn it OFF
- Works over the same USB cable used to upload sketches
- Send commands from the Arduino IDE's Serial Monitor




# Arduino <-> PC: Overview

- Reads one incoming character per loop — '1' = ON, '0' = OFF
- Reports the LED's actual state back every 100ms

This sketch runs two jobs every loop: listen, and report. Serial.available() checks for an incoming command byte; a '1' character turns the built-in LED on, a '0' turns it off — note the single quotes, since Serial.read() returns the character's byte value, not the digit itself. Then, regardless of whether a command arrived, digitalRead() checks the LED's actual current state and Serial.println() reports it back as plain text, so typing '1' or '0' into the Serial Monitor and watching the status line update is enough to see the whole loop working end to end.

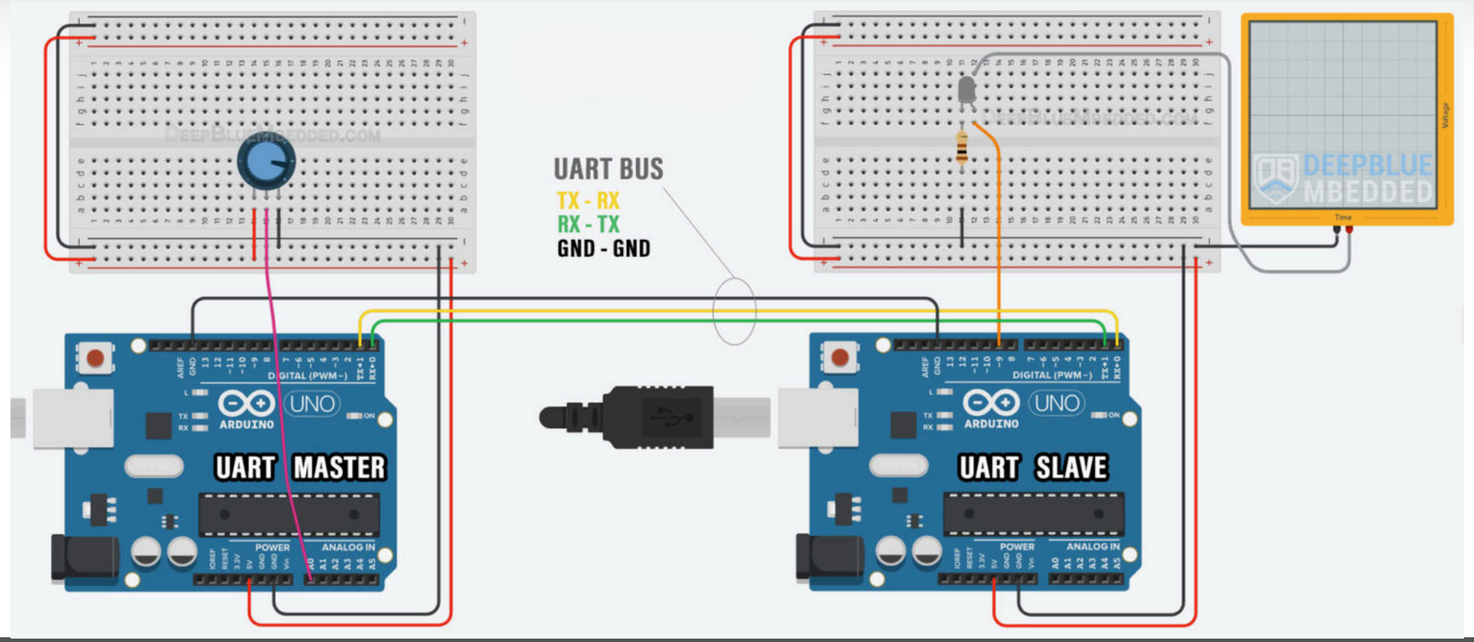
```
uint8_t incomingByte = 0;

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  if (Serial.available()) {
    incomingByte = Serial.read();
    if (incomingByte == '1') digitalWrite(LED_BUILTIN, 1);
    if (incomingByte == '0') digitalWrite(LED_BUILTIN, 0);
  }
  if (digitalRead(LED_BUILTIN))
    Serial.println("LED is ON");
  else
    Serial.println("LED is OFF");
  delay(100);
}
```

# Arduino <-> PC: Code

- One-way link: a Master Arduino talks, a Slave Arduino listens
- Master reads a potentiometer on A0 and sends its value over UART
- Slave receives each byte and uses it to set a PWM LED brightness
- Master's TX wires to Slave's RX (both share a common GND)
- The same pattern works for any sensor-to-actuator link between two boards




# Parity, Stop Bits & Flow Control

- Reads the 10-bit potentiometer, shrinks it to a UART-sized byte

```arduino
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.write(analogRead(A0) >> 2);
  delay(100);
}
```

analogRead(A0) returns a 10-bit value, 0 to 1023 — too wide to fit in the single byte UART sends at a time. The >> 2 shift divides it by 4, squeezing it down into the 0-255 range a byte can hold, and Serial.write() sends that raw byte straight out over TX. There's no text formatting here at all, unlike Serial.print() — write() ships the actual binary value, which is exactly what the slave board is set up to expect on the other end.

# Arduino <-> Arduino: Master

- Every incoming byte directly becomes the LED's new PWM duty cycle

```ino
#define LED_PIN 9
uint8_t incomingByte = 0;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  if (Serial.available()) {
    incomingByte = Serial.read();
    analogWrite(LED_PIN, incomingByte);
  }
  delay(100);
}
```

The slave's loop is almost the same read-and-react pattern as the PC example, just with a different reaction: whatever byte arrives from Serial.read() gets handed straight to analogWrite() as the new duty cycle. Because the master already scaled its reading down to 0-255 before sending, no further conversion is needed here — turning the potentiometer on the master board changes the LED's brightness on the slave board in near real time.

# Arduino <-> Arduino: Slave